COMP0005 – ALGORITHMS

Week 7 – 04 March 2025

Prof Licia Capra

Dept of Computer Science

OUTLINE

- Week Preview
- Muddiest Point Feedback
- Topic 3 (Search) Revision
- Exercises
- Coursework Overview

OUTLINE

- Week Preview
- Muddiest Point Feedback
- Topic 3 (Search) Revision
- Exercises
- Coursework Overview

WEEK PREVIEW

- Topic 4 – Graphs
  - 4.1 Undirected graphs
  - 4.2 Directed graphs
- Self-practice: Undirected and directed graphs
- Lab session: TA Lab – Week 7

OUTLINE

- Week Preview
- Muddiest Point Feedback
- Topic 3 (Search) Revision
- Exercises
- Coursework Overview

MUDDIEST POINT FEEDBACK

Question

Could you go over the time complexity of hash tables? I am not sure where the logarithmic worst cases come from. Also, the video has the worst cases of hash tables with linear probing as lg N, but the slides have them as N. An overall explanation of where they come from would be helpful.

Posted by Anonymous at Tuesday, 25 February 2025, 6:58 PM (18 hours 20 mins ago)

The pseudocode behind implementing an insert function into a red black BST. How the full function works recursively alongside having a regular BST search implemented within the function

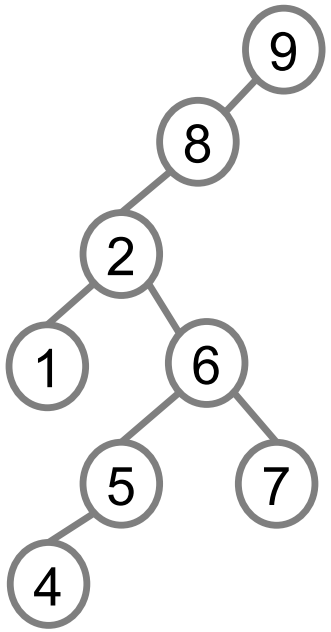
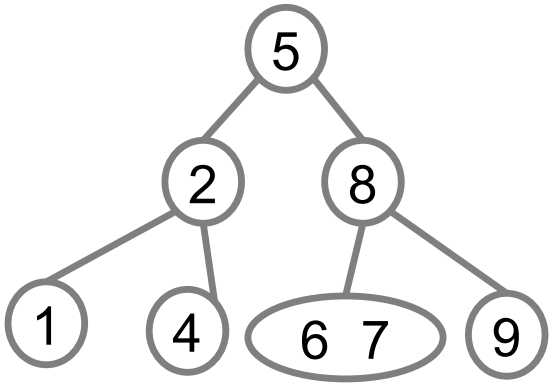
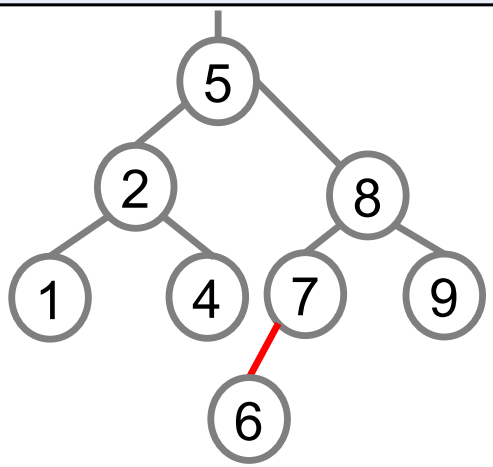
Posted by Anonymous at Tuesday, 25 February 2025, 1:11 PM (1 day ago)

OUTLINE

- Week Preview
- Muddiest Point Feedback
- Topic 3 (Search) Revision
- Exercises
- Coursework Overview

SEARCH TREES COMPARISON

Insert keys: 9, 8, 2, 6, 1, 5, 4, 7

Binary Search Tree (BST)	**Balanced** Search Tree	
	2-3 Search Tree	Left-Leaning Reb-Black (LLRB)
		

OUTLINE

- Week Preview
- Muddiest Point Feedback
- Topic 3 (Search) Revision
- Exercises
- Coursework Overview

EXERCISES – EXAM #1

Consider the following sequence of keys:

X V T R P N

- Illustrate the content of a Binary Search Tree (BST) after inserting all keys above in the given order. *[6 marks]*

- Illustrate the content of a Left Leaning Red Black (LLRB) binary search tree after inserting all keys above in the given order. Clearly mark which edges are coloured red, and which edges are coloured black. *[15 marks]*

EXERCISES – EXAM #1

Consider the following sequence of keys:

X V T R P N

```
def put(n, key, value):

    if (n == null) return Node(key, value, RED)

    # traditional BST put operation to locate where to insert new node

    if (isRed(n.right) && !isRed(n.left)) n = rotateLeft(n)
    if (isRed(n.left) && isRed(n.left.left)) n = rotateRight(n)
    if (isRed(n.left) && isRed(n.right)) flipColors(n)

    return n
```

insert at bottom and color it RED **(base case)**

recursive tree traversal to locate where to insert new (key,value) pair

make left leaning

balance 4-node

split 4-node

EXERCISES – EXAM #1

Consider the following sequence of keys:

X V T R P N

Traverse LLRB BST **down**

Traverse LLRB BST **up**

```
1. Perform a standard BST insert
2. Colour the new node RED
3. Repeat as necessary moving up the tree
   a. Rotate Left to make tree left leaning
   b. Rotate Right to balance 4-node
   c. Flip Colour to split 4-node
```

EXERCISES – EXAM #2

Consider a hash table of length 10 (indexed 0 to 9) that uses **separate chaining** and hash function `h(k)=k mod 10`. Illustrate the content of the hash table after inserting the following keys in the given order:

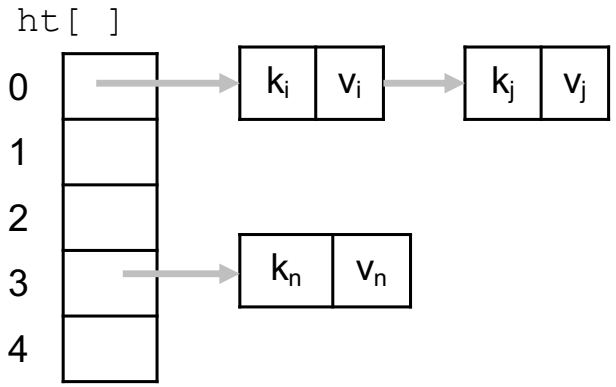
`44, 45, 74, 64, 55, 12, 15, 55, 22, 2`

Repeat the exercise, this time considering a hash table of length 20 (indexed 0 to 19) that uses **linear probing** instead.

[10 marks]

EXERCISES – EXAM #2

Separate chaining



Linear probing

	0	1	2	3	4	5	6	7	8	9	10
ht_keys[]	k_i	k_j		k_n							
ht_values[]	v_i	v_j		v_n							

EXERCISES – EXAM #2

	Worst Case (after N inserts)			Average Case (after N random inserts)		
	Search	Insert	Delete	Search (hit)	Insert	Delete
Hash table with separate chaining (*)	lg N	lg N	lg N	3-5	3-5	3-5
Hash table with linear probing (*)	N	N	N	3-5	3-5	3-5

(*) under uniform hashing assumptions

via self-balancing search tree (e.g., LLRB BST)

OUTLINE

- Week Preview
- Muddiest Point Feedback
- Topic 3 (Search) Revision
- Exercises
- Coursework Overview

COURSEWORK OVERVIEW

- Overview: Experimental Evaluation of Search Algorithms

	Data Structure	**Space** Complexity	**Time** Complexity for **Search** Op (worst case)
[1]	2-3 Tree	*O(n)*	*O(log n)*
[2]	AVL Tree		
[3]	LLRB BST		
[4]	B-Tree		
[5]	Scapegoat Tree		

- Task:
  - to create an experimental evaluation framework and use it to comprehensively evaluate three data structures of your choice out of the five listed above;
  - to thoroughly and critically discuss the circumstances under which some data structures are better suited than others (among the three you have chosen to compare).

COURSEWORK OVERVIEW

- Follow the instructions/constraints carefully:
  - Jupyter Notebook with Python 3.11
  - Libraries: `string`, `random`, `timeit`, and `matplotlib`
  - Use the `.ipynb` template provided for the code
  - Conduct experiments on your local machine (not on the cloud)
  - Follow the constraints on the report

COURSEWORK OVERVIEW

```python
# DO NOT MODIFY THIS CELL

from abc import ABC, abstractmethod

class AbstractSearchInterface(ABC):
    '''
    Abstract class to support search/insert operations (plus underlying data structure)

    '''

    @abstractmethod
    def insertElement(self, element):
        '''
        Insert an element in a search tree
            Parameters:
                    element: string to be inserted in the search tree (string)

            Returns:
                    "True" after successful insertion, "False" if element is already present (bool)
        '''

        pass


    @abstractmethod
    def searchElement(self, element):
        '''
        Search for an element in a search tree
            Parameters:
                    element: string to be searched in the search tree (string)

            Returns:
                    "True" if element is found, "False" otherwise (bool)
        '''

        pass
```

COURSEWORK OVERVIEW

```python
class TwoThreeTree(AbstractSearchInterface):

    def insertElement(self, element):
        inserted = False
        # ADD YOUR CODE HERE


        return inserted



    def searchElement(self, element):
        found = False
        # ADD YOUR CODE HERE


        return found
```

```python
class LLRBBST(AbstractSearchInterface):

    def insertElement(self, element):
        inserted = False
        # ADD YOUR CODE HERE


        return inserted



    def searchElement(self, element):
        found = False
        # ADD YOUR CODE HERE


        return found
```

```python
class AVLTree(AbstractSearchInterface):

    def insertElement(self, element):
        inserted = False
        # ADD YOUR CODE HERE


        return inserted



    def searchElement(self, element):
        found = False
        # ADD YOUR CODE HERE


        return found
```

```python
class BTree(AbstractSearchInterface):

    def insertElement(self, element):
        inserted = False
        # ADD YOUR CODE HERE


        return inserted



    def searchElement(self, element):
        found = False
        # ADD YOUR CODE HERE


        return found
```

```python
class ScapegoatTree(AbstractSearchInterface):

    def insertElement(self, element):
        inserted = False
        # ADD YOUR CODE HERE


        return inserted



    def searchElement(self, element):
        found = False
        # ADD YOUR CODE HERE


        return found
```

```python
import string
import random

class TestDataGenerator():
    """
    A class to represent a synthetic data generator.

    ...

    Attributes
    ----------

    [to be defined as part of the coursework]

    Methods
    -------

    [to be defined as part of the coursework]

    """

    #ADD YOUR CODE HERE

    def __init__():
        pass
```

```python
import timeit
import matplotlib

class ExperimentalFramework():
    """
    A class to represent an experimental framework.

    ...

    Attributes
    ----------

    [to be defined as part of the coursework]

    Methods
    -------

    [to be defined as part of the coursework]

    """

    #ADD YOUR CODE HERE

    def __init__():
        pass
```

```python
# ADD YOUR TEST CODE HERE
```

Experimental analysis: 60% of coursework mark

COURSEWORK OVERVIEW

COMP0005 Group Coursework
Academic Year 2024-25
Group Number: XXX

1 Overview of Experimental Framework
1.1 Framework Design/Architecture
1.2 Hardware/Software Setup for Experimentation
(Section 1 should be about one page)

2 Performance Results
2.1 [First Chosen Data Structure]
2.2 [Second Chosen Data Structure]
2.3 [Third Chosen Data Structure]
(Section 2 should be about two / two-and-a-half pages)

3 Comparative Assessment

(Section 3 should be about one / one-and-a-half pages)

4 Team Contributions

Student Name	Student Portico ID	Key Contributions	Share of work[1]
			… %
			… %
			… %
			… %

(Section 4 should be no more than half a page)

Experimental analysis:

60% of coursework mark

COURSEWORK OVERVIEW

- Deadline: March 24th @ 4pm UK time

- Each group submits 2 files:
  - Jupyter notebook `grp-[your group number]-0005code.ipynb`
  - PDF report `grp-[your group number]-0005report.pdf`

COURSEWORK OVERVIEW

- Coursework: 20% of COMP0005 final mark

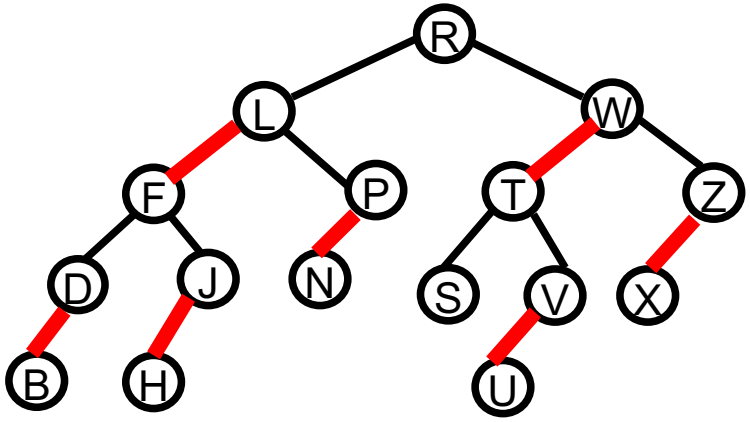
- Coursework assessment criteria:
  - Quality of code [40 marks]

    (i.e., is your implementation correct? Is your code readable, well documented, reusable? Is it time and space efficient?)
  - Experimental analysis: [60 marks]

    (e.g., is your experimental framework comprehensive? Have edge-cases been analysed? Have suitable stress-test conditions been experimented with? Are the results of your experimentation correct? Are they thoroughly discussed?)

EXERCISES – EXAM #3

Consider the following left-leaning red-black (LLRB) BST rooted in R:



- How many “colour flips”, “left rotations” and “right rotations” take place to insert key Q? What key appears in the root node after the insertion of Q? [3 marks]
- How many “colour flips”, “left rotations” and “right rotations” take place to insert key A (assume Q has NOT been inserted)? What key appears in the root node after the insertion of A? [3 marks]
- How many “colour flips”, “left rotations” and “right rotations” take place to insert key Y (assume neither Q nor A have been inserted)? What key appears in the root node after the insertion of Y? [3 marks]

EXERCISES – EXAM #4

Suppose you have a hash function that maps string keys to integers as follow:

key		hash(key)
Doc	→	1
Grumpy	→	5
Happy	→	6
Sleepy	→	0
Bashful	→	1
Sneezy	→	6
Dopey	→	4

0	
1	
2	
3	
4	
5	
6	

0	Sneezy
1	Sleepy
2	Bashful
3	Doc
4	Dopey
5	Grumpy
6	Happy

Imagine using *linear probing* to insert all the above keys in a hash table of size 7 that is initially empty.

Q: Could **Doc** have been the last inserted key to create the hash table above?

EXERCISES – EXAM #4

Suppose you have a hash function that maps string keys to integers as follow:

key		hash(key)
Doc	→	1
Grumpy	→	5
Happy	→	6
Sleepy	→	0
Bashful	→	1
Sneezy	→	6
Dopey	→	4

0	
1	
2	
3	
4	
5	
6	

0	Sneezy
1	Sleepy
2	Bashful
3	Doc
4	Dopey
5	Grumpy
6	Happy

Imagine using *linear probing* to insert all the above keys in a hash table of size 7 that is initially empty.

Q: Could **Sleepy** have been the last inserted key to create the hash table above?

EXERCISES – EXAM #4

Suppose you have a hash function that maps string keys to integers as follow:

key		hash(key)
Doc	→	1
Grumpy	→	5
Happy	→	6
Sleepy	→	0
Bashful	→	1
Sneezy	→	6
Dopey	→	4

0	
1	
2	
3	
4	
5	
6	

0	Sneezy
1	Sleepy
2	Bashful
3	Doc
4	Dopey
5	Grumpy
6	Happy

Imagine using *linear probing* to insert all the above keys in a hash table of size 7 that is initially empty.

Q: Could **Bashful** have been the last inserted key to create the hash table above?